BİL 468 / BİL 587 – 3. Ödev Raporu

Erişim Bilgileri

Orijinal Veri Seti Adı: detection-datasets/fashionpedia
Orijinal Veri Seti Bağlantısı: https://huggingface.co/datasets/detection-datasets/fashionpedia
Repo Bağlantısı: https://github.com/Zehraatalay/stylematch/tree/main/HW3
Ödev Çalışmasının Yüklendiği Google Drive Bağlantısı: https://drive.google.com/drive/folders/1qkBcfSOlh5R5hstf3DQ25nrPnYsD2K3E

Çalıştırılacak Dosyalar ve Ne Yaptıkları:

- ktree.py: K-Tree çekirdek mantığı — hierarchical K-Means ile tree oluşturma ve leaf prediction; class frequency'ye göre tamamen normalize edilmiş voting (norm_power=1.0) ve az örnekli/gürültülü leaf'leri eleyen minimum support threshold (min_leaf_support=5)
- metrics.py: Confusion matrix, precision, recall, F1-score raporu ve PCA — NumPy ile manuel olarak uygulanır; sklearn.metrics ve sklearn.decomposition kullanılmaz
- data_prep.py: Fashionpedia'dan class başına orantılı random sampling yapılır. Train ve validation setleri aynı Hugging Face train split'inden, birbirleriyle overlap olmayacak şekilde oluşturulur. Test seti Hugging Face val split'inden alınır; bu split'te kotayı karşılayamayan class'lar için kullanılmamış train image'larından ek sampling yapılır
- train.py: K-Tree modelini eğitir, training time bilgisini kaydeder ve modeli ktree_model.pkl olarak kaydeder
- detect_eval.py: SIFT descriptor'ları image başına bir kez çıkarılır ve sliding window'lara göre filtrelenir. 128/64/32/16 boyutlarında, %50 overlap ile multi-scale scanning uygulanır. Ground-truth label, IoU threshold'a göre belirlenir. Class prediction, choose_class fonksiyonunda margin ve min_object_prob threshold'ları kullanılarak yapılır. Confusion matrix ve class-based metrics hesaplanır.
- tune_pca.py: Farklı K ve D değerleri için hyperparameter sweep gerçekleştirir. Her configuration, training time ve validation macro F1-score üzerinden değerlendirilir. En yüksek macro F1-score'a sahip model otomatik olarak kaydedilir. K-Tree leaf center'ları PCA ile 2D olarak görselleştirilir.
- show_test.py: Test setindeki en iyi ve en kötü detection performance'a sahip image'ları otomatik seçer ve predicted/ground-truth bounding box'ları kullanarak görselleştirir.
- Dosyaların çalıştırılma sırası : 1- python data_prep.py 2- python tune_pca.py --iou 0.3 3- python detect_eval.py --iou 0.3 4- python show_test.py --iou 0.3

Veri Seti ve Değerlendirme Metrikleri

- Seçilen class'lar (en büyük 10): sleeve, shoe, neckline, pocket, dress, top/t-shirt/sweatshirt, pants, collar, zipper, jacket
- Image boyutu: 128×128 (grayscale)
- Sliding window yapısı: sırasıyla 128, 64, 32 ve 16 boyutlarında window, stride = window/2 (%50 overlap)
- IoU threshold: 0.3 (yalnızca evaluation aşamasında kullanılır; training descriptor'larının labeling işlemi point-in-box kontrolüyle yapılır.)

- Train: 10000 image, 966435 descriptor (class başına image sayıları: sleeve 6735, shoe 5362, neckline 7056, pocket 3384, dress 3935, top/t-shirt/sweatshirt 3606, pants 3096, collar 2576, zipper 1535, jacket 2204)
- Validation: 654 image, 63916 descriptor (tüm class'lar ≥200 image hedefini karşıladı)
- Test: Hugging Face val split'inden 590 image + zipper class'ının 200 image kotasını karşılaması için kullanılmamış train image'larından 103 ek image = 693 image, 70300 descriptor. Zipper dahil tüm class'lar final test setinde ≥200 image hedefine ulaştı (zipper tam sınırda: 200).
- Evaluation metrics: accuracy, class-based precision/recall/F1-score, confusion matrix (11×11, background dahil), macro precision/recall/F1-score (background hariç). Tüm metrikler NumPy kullanılarak manuel olarak hesaplanmıştır.

Yöntem

Training image'lar 128×128 grayscale olarak hazırlanmış, SIFT descriptor'ları bounding box bilgisine göre etiketlenmiştir. K-Tree, hierarchical K-Means ile K ve D parametreleri kullanılarak oluşturulmuştur. Evaluation aşamasında SIFT her image için yalnızca bir kez çıkarılmış, sliding window'lar descriptor filtering ile değerlendirilmiştir. Class prediction, norm_power=1.0, min_leaf_support=5, min_object_prob=0.20 ve min_margin=0.08 parametreleri kullanılarak yapılmıştır. Ground-truth label ise en yüksek IoU (threshold = 0.3) değerine göre atanmıştır.

Nihai K-Tree Yapısı

Hiperparametre taramasında doğrulama setinde en yüksek macro F1'i veren kombinasyon otomatik olarak seçilip kaydedilmiştir:

- Seçilen K: 16
- Seçilen D: 4
- Yaprak sayısı: 65536
- Eğitim süresi: ~85.4 saniye

Deney Sonuçları (Hyperparameter Sweep)

Doğrulama seti üzerinde K,D taraması (IoU=0.3, margin=0.08, min_object_prob=0.20):

K: Her aşamadaki küme sayısı (braching facator)
D: Ağaç derinliği

K	D	Yaprak Sayısı	Eğitim Süresi (s)	Val Accuracy	Val Macro Precision	Val Macro Recall	Val Macro F1
4	2	16	10.96	0.951	0.000	0.000	0.000

4	3	64	15.90	0.951	0.000	0.000	0.000
4	4	256	25.09	0.951	0.000	0.000	0.000
8	2	64	20.11	0.951	0.000	0.000	0.000
8	3	512	33.88	0.951	0.000	0.000	0.000
8	4	4096	53.19	0.948	0.010	0.001	0.002
16	2	256	41.69	0.951	0.000	0.000	0.000
16	3	4096	57.85	0.948	0.010	0.002	0.003
16	4	65536	85.38	0.916	0.016	0.013	0.013

Gözlem : Hyperparameter sweep sonuçları, K ve D arttıkça training time'ın belirgin şekilde yükseldiğini göstermektedir. Buna karşın doğrulama başarımı (macro F1) düşük değerlerde kalmış ve yalnızca en büyük tree yapılarında sınırlı bir artış gözlenmiştir. Özellikle K=4 ve K=8, D≤3 kombinasyonlarında model tüm region'ları background olarak tahmin etmiş (macro F1 = 0.000). İlk anlamlı iyileşme K=8, D=4 ve K=16, D=3 kombinasyonlarında görülmüş, en yüksek başarı ise K=16, D=4 (macro F1 ≈ 0.013) ile elde edilmiştir. Bu sonuçlar, daha büyük K-Tree yapılarını kullanmanın başarımı bir miktar artırmasına rağmen, hesaplama maliyetini çok daha hızlı artırdığını ve performans kazancının sınırlı kaldığını göstermektedir.

Test Sonuçları

Nihai model (K=16, D=4) ile test setinde elde edilen sonuçlar (IoU=0.3, margin=0.08, min_object_prob=0.20):

- Genel accuracy: 0.921 (büyük ölçüde arka plan sınıfının payından kaynaklanır, support=187653/~199812 ≈ %94)
- Macro precision (arka plan hariç, 10 nesne sınıfı): 0.0085
- Macro recall (arka plan hariç): 0.0068
- Macro F1 (arka plan hariç): 0.0072

Sınıf	**Precision**	**Recall**	**F1**	**Support**
background	0.953	0.966	0.960	187653
sleeve	0.022	0.006	0.009	2813
shoe	0.042	0.038	0.040	1249

neckline	0.000	0.000	0.000	366
pocket	0.000	0.000	0.000	310
dress	0.002	0.002	0.002	863
top, t-shirt, sweatshirt	0.000	0.000	0.000	1038
pants	0.015	0.016	0.016	1073
collar	0.004	0.006	0.005	319
zipper	0.000	0.000	0.000	200
jacket	0.000	0.000	0.000	928

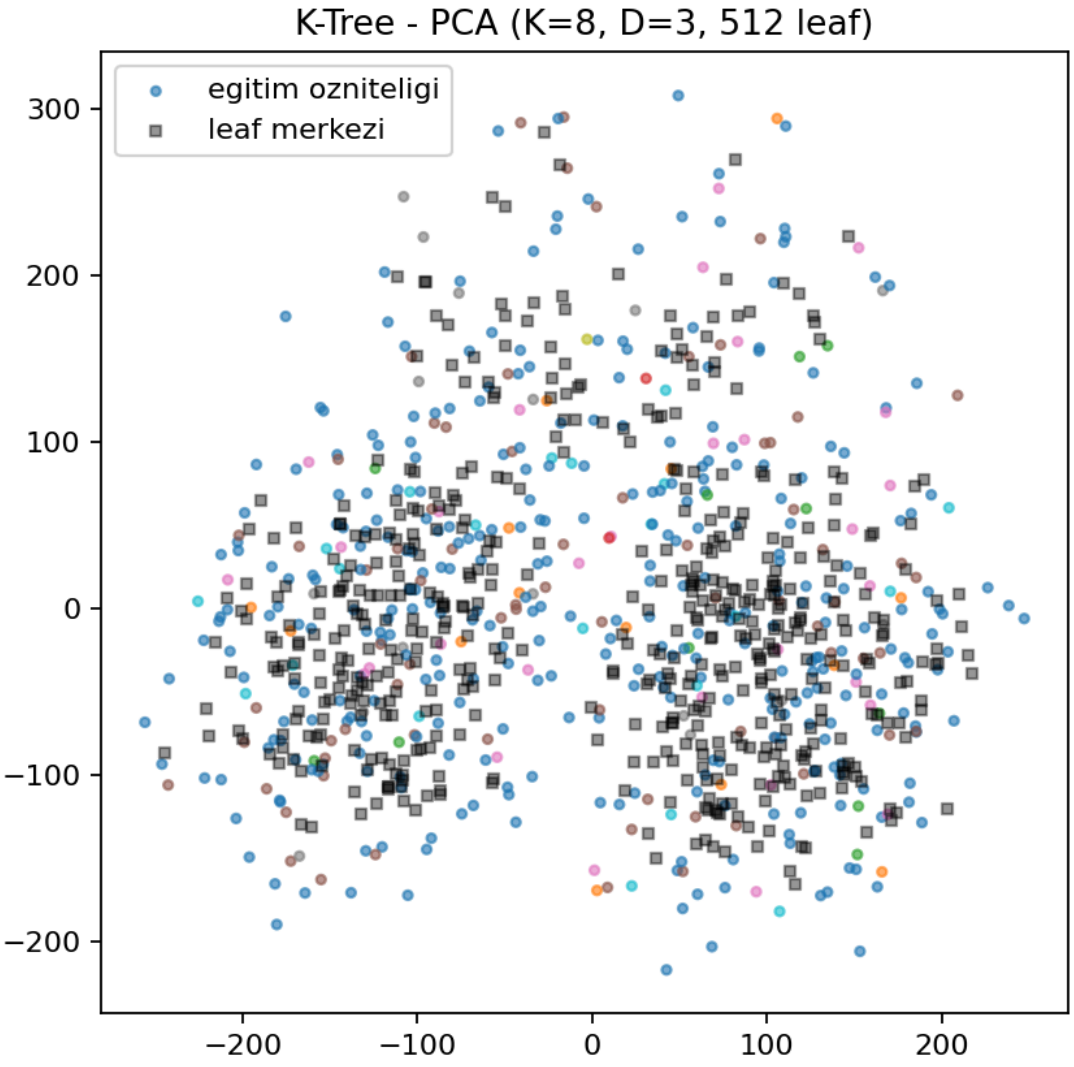
Test Confusion Matrix

Satırlar gerçek sınıf, sütunlar tahmin edilen sınıf (0=background, 1=sleeve, 2=shoe, 3=neckline, 4=pocket, 5=dress, 6=top/t-shirt/sweatshirt, 7=pants, 8=collar, 9=zipper, 10=jacket):

GT \ Pred	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
0	181246	696	1037	482	350	1120	385	1052	444	472	369
1	2679	16	13	15	18	22	3	24	3	12	8
2	1163	4	47	4	0	5	0	17	1	3	5
3	356	0	0	0	1	4	1	1	0	1	2
4	295	1	1	2	0	1	3	2	1	0	4
5	860	0	0	0	0	2	0	1	0	0	0
6	1027	1	0	1	1	2	0	2	1	1	2
7	1047	0	8	0	1	0	0	17	0	0	0
8	312	0	0	1	1	2	0	1	2	0	0
9	192	2	0	0	0	4	0	1	0	0	1
10	918	1	3	0	1	3	0	2	0	0	0

Yorum: Yanlış tahminlerin büyük çoğunluğu background sütununda toplanmıştır; model gerçek object'leri de çoğunlukla background olarak sınıflandırmaktadır (düşük recall). Köşegendeki doğru tahmin sayıları ise oldukça düşüktür (örn. shoe: 47, pants: 17, sleeve: 16).

PCA Görselleştirmesi

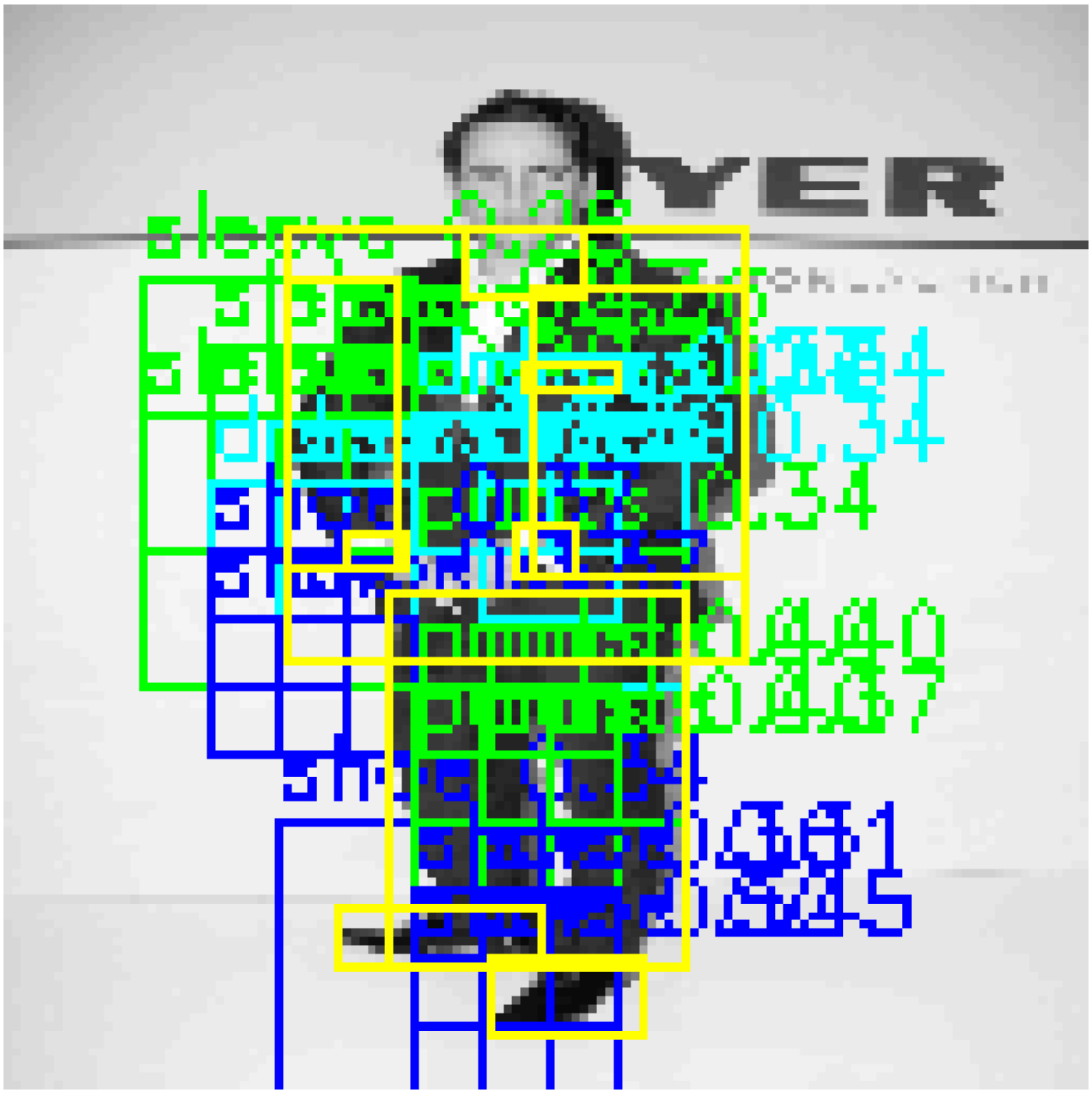


Yorum: PCA görselleştirmesi, leaf center'larının feature space boyunca dağıldığını ancak farklı class'lara ait training descriptor'ların büyük ölçüde overlap ettiğini göstermektedir. Belirgin bir class separation oluşmaması, modelin object class'larını ayırt etmekte zorlanmasını ve düşük macro F1 performansını açıklamaktadır.

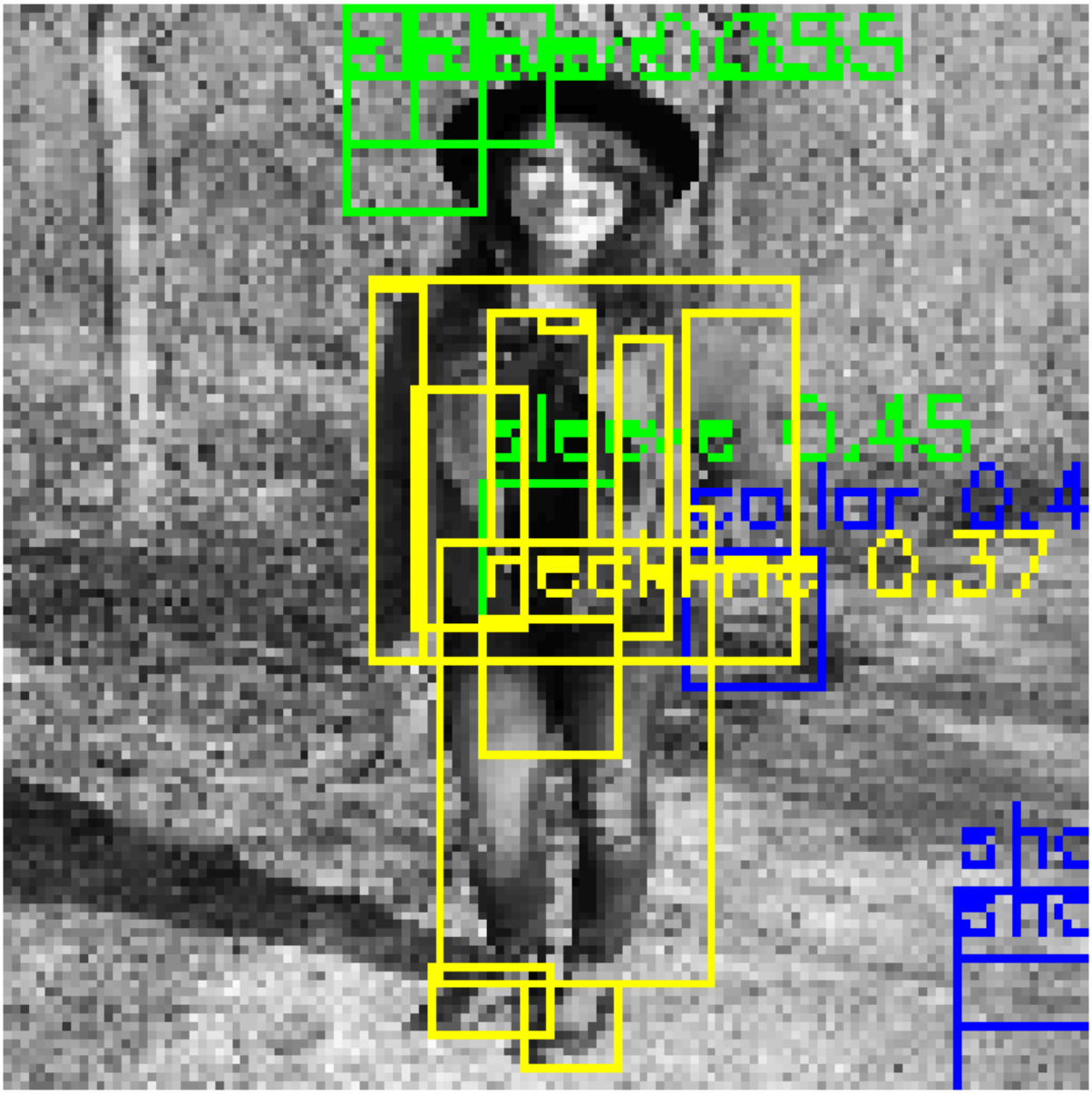
Örnek Tespit Görselleri (İyi/Kötü Örnekler)

show_test.py, test setindeki gerçek-pozitif pencerelerin ne kadarının doğru sınıflandırıldığına göre en iyi ve en kötü 3'er örneği otomatik seçmiştir:

Dosya	**Doğru / Toplam Gerçek-Pozitif Pencere**	**Oran**
test_example_good_0.png	7 / 22	%32
test_example_good_1.png	4 / 14	%29
test_example_good_2.png	2 / 8	%25
test_example_bad_0.png	0 / 21	%0
test_example_bad_1.png	0 / 9	%0
test_example_bad_2.png	0 / 12	%0



- Good example



- Bad example

Good Example: Model, nesnenin bulunduğu bölgeyi kısmen yakalayabilmiş olsa da çok sayıda false positive window üretmiş ve tahminler farklı class'lara dağılmıştır. Bu örnek, en başarılı sonuçlardan biri olmasına rağmen doğru tahmin oranı yalnızca %25–32 seviyesindedir.

Bad Example: Model, nesne üzerinde çok sayıda yanlış bounding box üretmiş ve farklı class tahminleri yapmıştır. Doğru bölgeyi tutarlı şekilde belirleyemediği için false positive sayısı yüksektir ve detection başarımı oldukça düşüktür.

Sonuç ve Değerlendirme

Test setinde accuracy=0.921 elde edilmiştir, ancak bu neredeyse tamamen background sınıfının baskınlığından kaynaklanır (support'un ~%94'ü). 10 nesne sınıfının ortalama F1'i yalnızca 0.007'dir; 5 sınıfta (neckline, pocket, top/t-shirt/sweatshirt, zipper, jacket) hiç doğru tahmin yapılamamıştır. En iyi ayrışan sınıflar shoe (F1=0.040) ve pants (F1=0.016) olmuştur.

Hyperparameter taramasındaki en önemli bulgu: sinyal yalnızca ağaç çok derinleştiğinde (K=16,D=4, 65536 leaf) ortaya çıkmıştır — daha sığ ağaçlarda leaf'ler yeterince ayırt edici olmadığından model her zaman background tahmin etmiştir. Validation macro F1'in (0.013) test'tekinden (0.007) belirgin yüksek olması, K,D seçiminin validation üzerinden yapılmasından kaynaklanan hafif bir overfitting'e işaret etmektedir.

Sınırlı performansın olası nedenleri: 128x128 çözünürlüğün küçük parçaları (sleeve, collar, zipper gibi) neredeyse görünmez kılması, SIFT'in renksiz olması, seçilen sınıfların (jacket/coat/cardigan gibi) görsel benzerliği, ve norm_power=1.0 normalizasyonunun ayırt edici olmayan leaf'leri doğru nötrlese de ayırt edici leaf'lerin kendisinin nadir/noisy kalmasıdır